Containerlab overview

Network labs for the future

Nardus le Roux

Apr 2026 @ ZANOG26

CONTAINERlab

Agenda

1. The lab as code approach
2. How it works
3. New tools

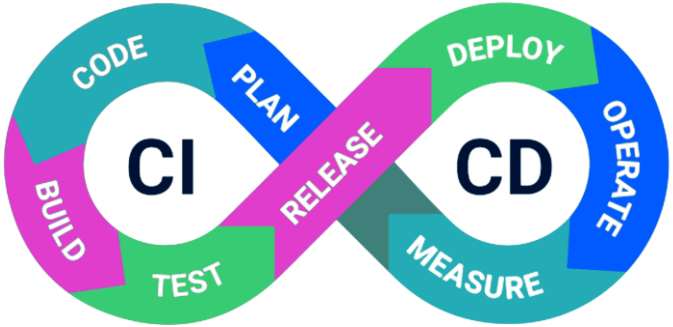
Agenda

1. The lab as code approach
2. How it works
3. New tools

CONTAINERlab

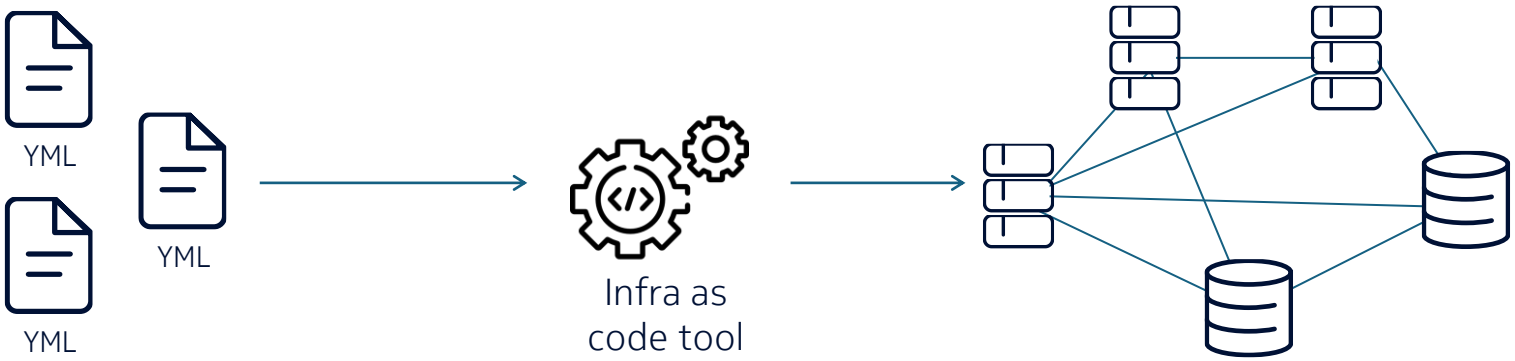
Virtual labs
A right, not a privilege

Education
and learning



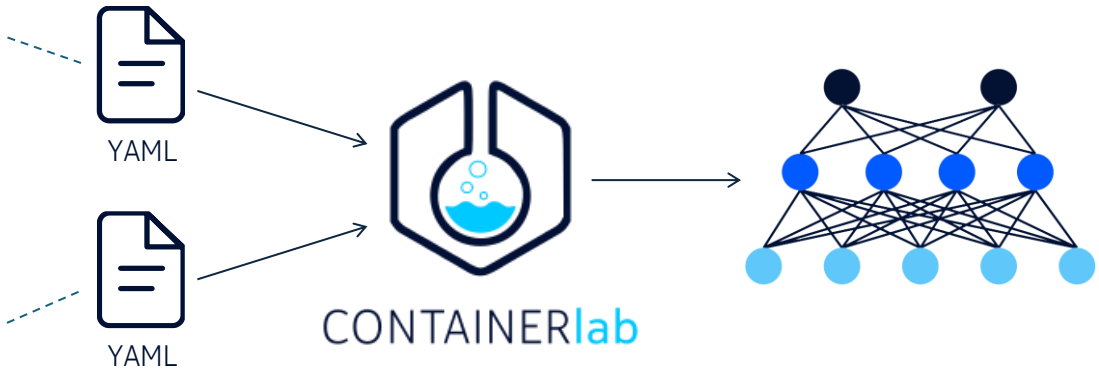
Change management
and validation

How do application teams (typically) deploy the services? Declarative way



Bringing declarativeness to network labs

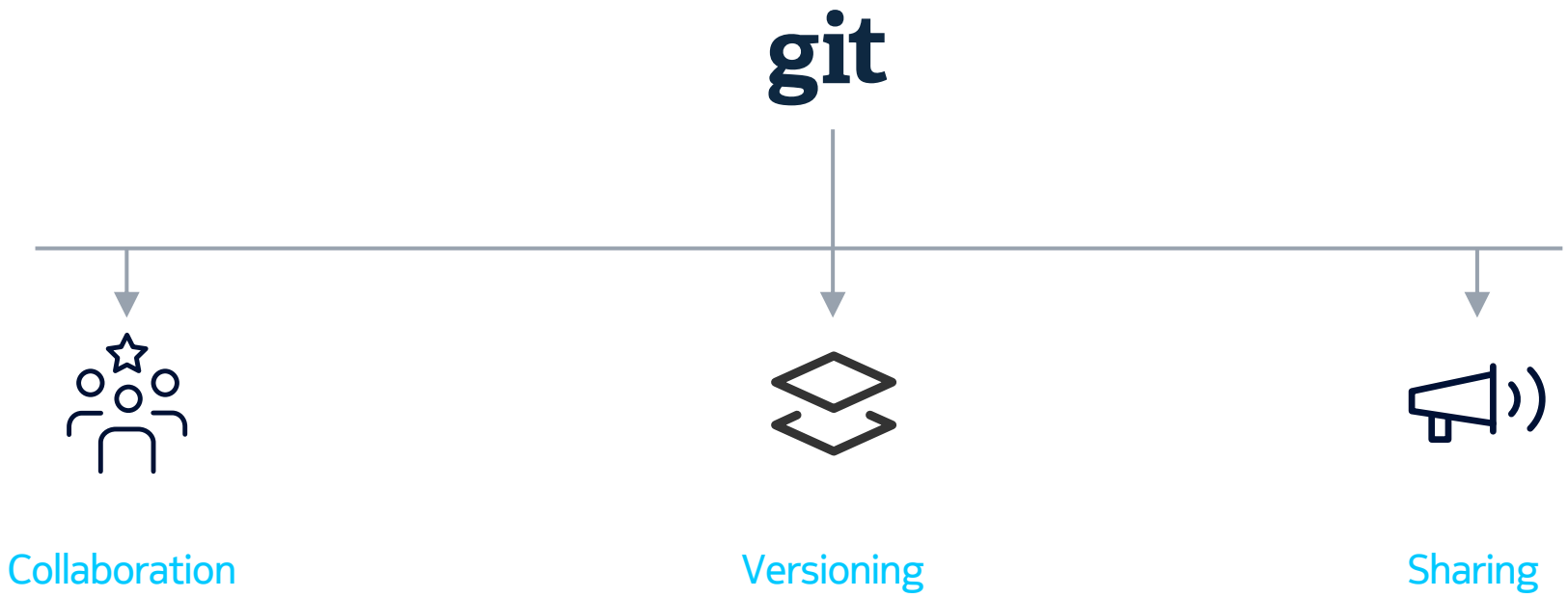
```yaml
name: mylab

topology:
  nodes:
    [redacted]: [redacted]
    [redacted]: [redacted]

  links:
    - [redacted]: [redacted]
```



containerlab.dev

CONTAINERlab

Virtual labs “Okay”, but why “as Code”?



Containerlab node types
Containerized network OSes

- Sourced by the vendor
- Fast to spin up
- Small footprint
- Shareability and versioning

Current trend is to **move away from VM** packaging towards containers **for new NOSes**

NOKIA
SR Linux

Juniper Networks
cRPD

ARISTA
cEOS

cisco
XRd

NVIDIA
cVX

KEYSIGHT TECHNOLOGIES
IXIA-c

and others...

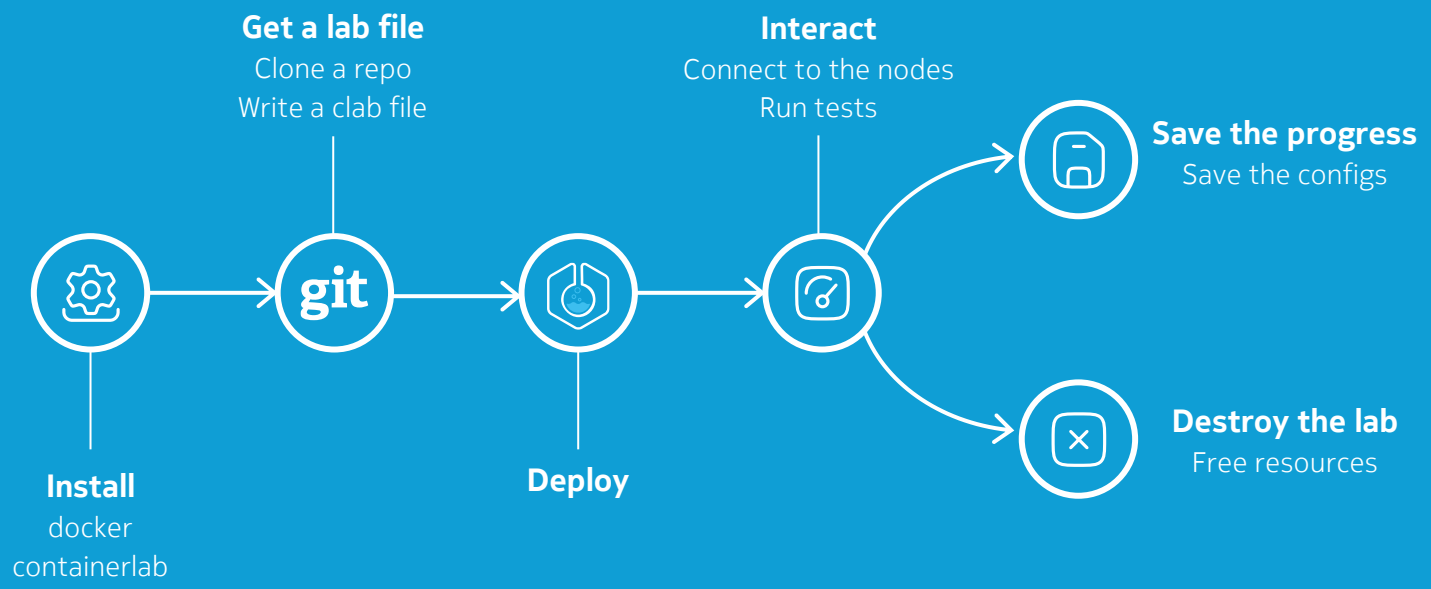
Containerlab node types
Regular container images for a complete ecosystem

- All available container images
- Emulating clients
- Hundreds of network-focused software
  - Telemetry, logging stacks
  - Peering software
  - Flow collectors
  - etc.

Agenda

1. The lab as code approach
2. How it works
3. New tools

Containerlab
The Workflow



Installation
In a few seconds

```
root@devbox:~
bash -c "$(curl -sL https://get.containerlab.dev)"
```

Installation options:
https://containerlab.dev/install/

Topology file
Writing your own topology

```yaml
name: my-lab

topology:
  nodes:
    router1:
      kind: nokia_sros
      image: sros:24.3.R1
      startup-config: r1.cfg
```

1. Node definition container.
   Container name will be the node name.
   Read more

2. Kinds define the flavour of the node, it says if the node is a specific containerized Network OS or something else.
   Read more

3. Image specifies container image to use for this node.
   Read more

4. Path to a startup configuration file (optional).
   Read more

topology definition file

Topology File
Logical view

topology definition

demo1.clab.yml

```yaml
name: demo1

topology:
  nodes:

    srl:
      kind: nokia_srlinux
      image: ghcr.io/nokia/srlinux:24.3.2

    sros:
      kind: nokia_sros
      image: sros:24.3.R2-1
      license: license-sros24.txt

  links:
    - endpoints: ["srl:e1-1", "sros:eth1"]
```

logical view

Containerlab CLI: Deploy
Deploy and access the nodes

```
$ containerlab deploy -t demo1.clab.yml
+---+----------------+--------------+-----------------------------+---------+---------+----------------+----------------------+
| # |      Name      | Container ID |            Image            |  Kind   |  State  |  IPv4 Address  |     IPv6 Address     |
+---+----------------+--------------+-----------------------------+---------+---------+----------------+----------------------+
| 1 | clab-demo1-srl | b241e0db0a79 | ghcr.io/nokia/srlinux:22.6.2 | srl     | running | 172.20.20.3/24 | 2001:172:20:20::3/64 |
| 2 | clab-demo1-sros | 62610c5026f3 | sros:21.10.R2               | vr-sros | running | 172.20.20.2/24 | 2001:172:20:20::2/64 |
+---+----------------+--------------+-----------------------------+---------+---------+----------------+----------------------+
```

SR Linux

```
ssh admin@clab-demo1-srl

Welcome to the srlinux CLI.
Type 'help' (and press <ENTER>) if
you need any help using this.
--{ running }--[  ]-
A:srl#
```

SR OS

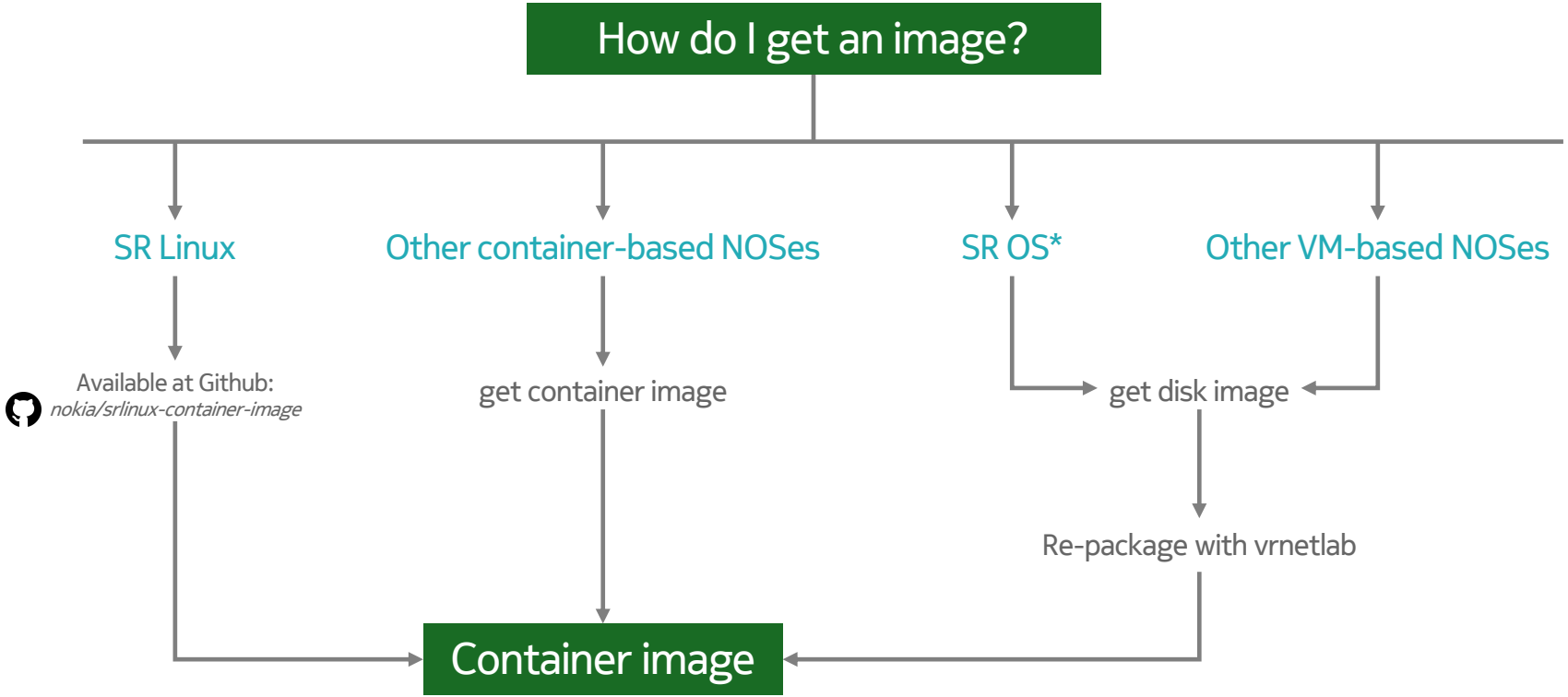
```
ssh admin@172.20.20.2

admin@172.20.20.2's password:

 SR OS Software
 Copyright (c) Nokia 2021.  All
Rights Reserved.
```

Containerlab CLI
Save

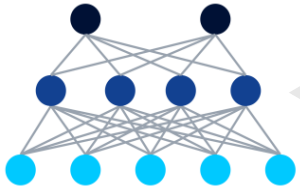
```
# executed in a directory where demo1.clab.yml is present
> containerlab save

INFO[0000] Parsing & checking topology file:
demo1.clab.yml
INFO[0000] saved sros running configuration to startup
configuration file
INFO[0001] saved SR Linux configuration from srl node.
```

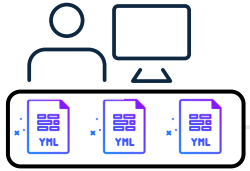
Perform lab-scoped configuration save in a single sweep

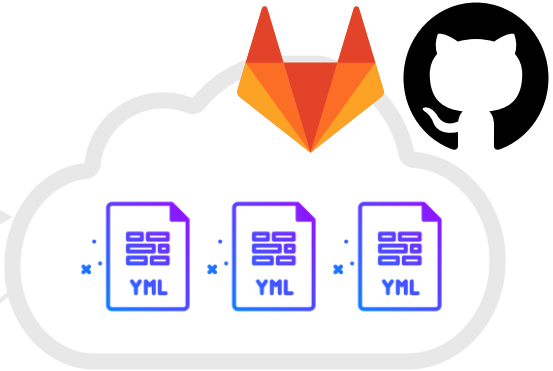
Container Images
Where do I get one?

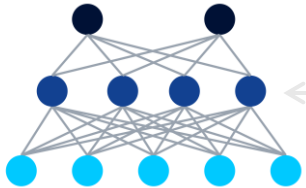


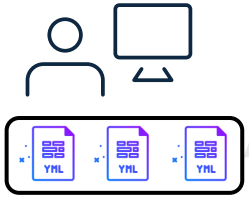
Share your labs
Lab as code

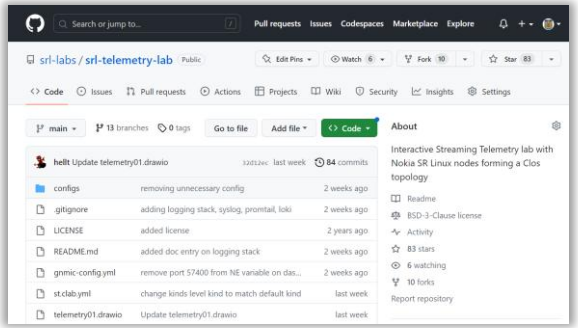




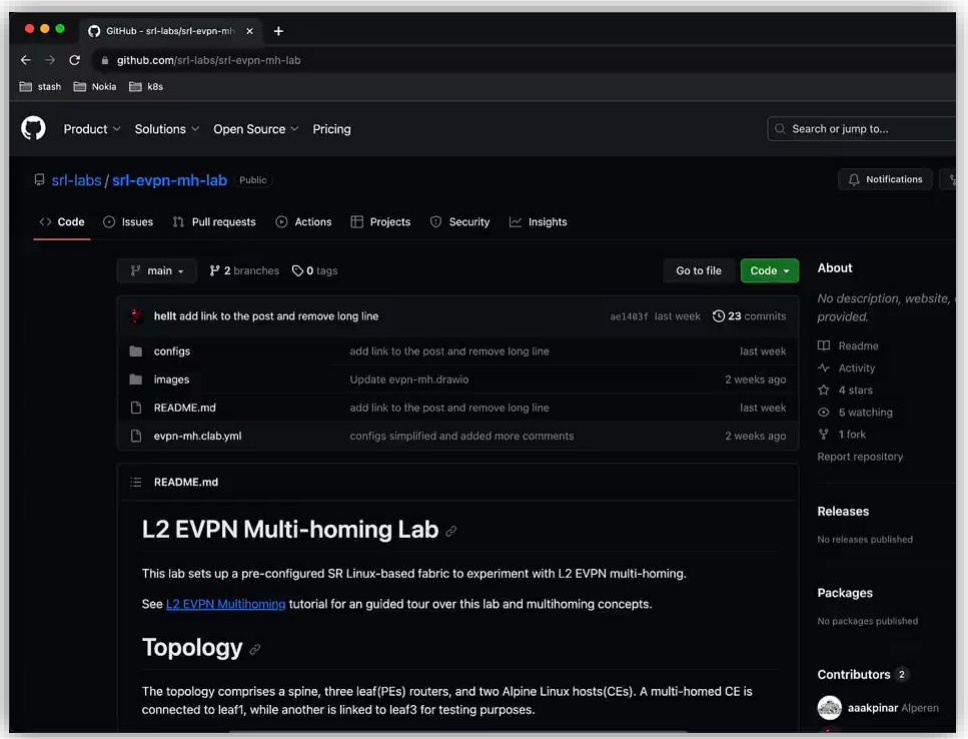






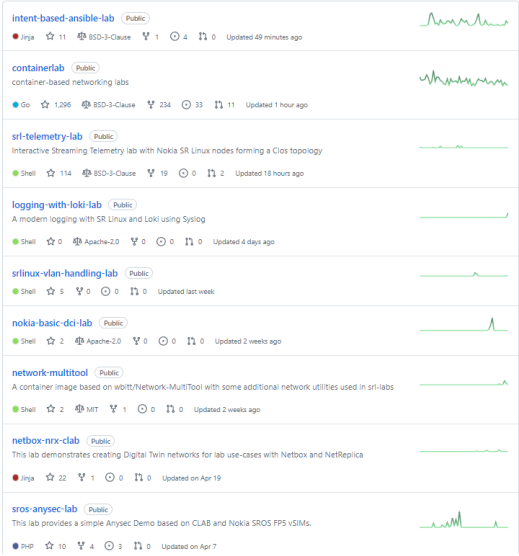


Containerlab CLI: Deploy Remote Deploy from github



- Eliminate the step of git clone:

```
$ clab deploy -t https://github.com/srl-labs/srl-evpn-mh-lab
```



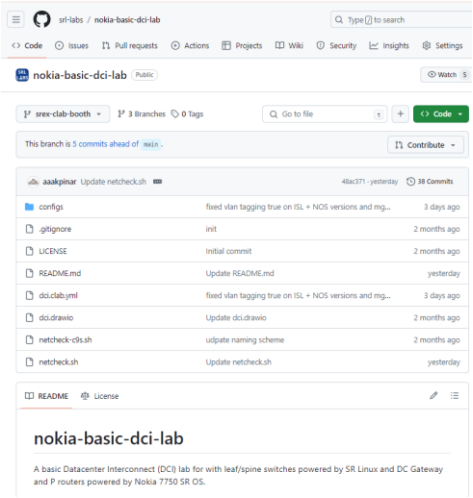
CONTAINERlab

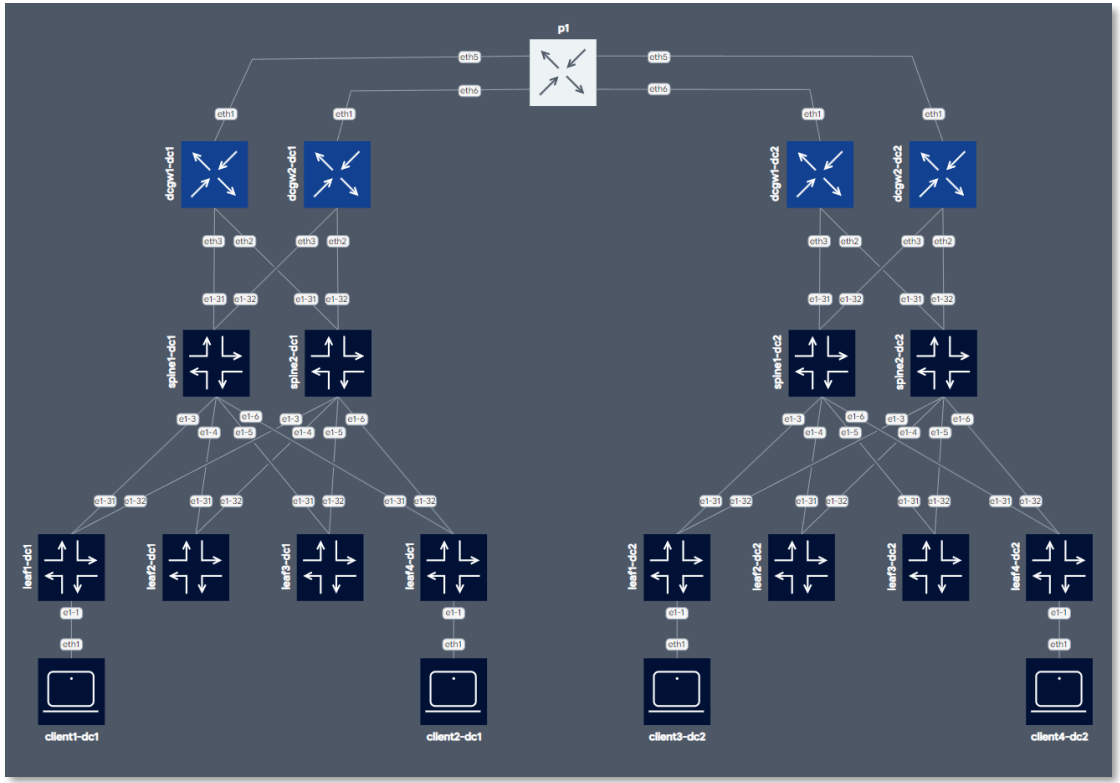
Deploying a complete lab

DC Interconnect example

DC Interconnect Lab Demo

- SRL & SROS mixed topology
- Explore DC interconnect scenarios
- DC Fabric and WAN (EVPN / VPNv4)
- Available on github.com/srl-labs





CONTAINERlab

DC Interconnect example
1. Deploy

```
ubuntu@ti2m:~/gitlab/nokia-basic-dci-lab$
```

CONTAINERlab

DC Interconnect example
2. Interact

```
ubuntu@ti2m:~/gitlab/nokia-basic-dci-lab$
```

Agenda

1. The lab as code approach
2. How it works
3. New tools

What's new?
Edgeshark integration

```
$ curl -sL \
https://github.com/siemens/edgeshark/raw/main/deployments/wget/docker-compose.yaml \
| DOCKER_DEFAULT_PLATFORM= docker compose -f - up -d
```

- Deploys Edgeshark containers
- Expose the Web UI on the containerlab's host port 5001
- You will also need the external capture plugin

What's new?
Deploy complete labs in Codespaces

- Containerlab simplifies the labbing experience
- But you still have to provide a VM

Not anymore with labs in Codespaces.

Run your labs in the cloud:

Nokia SR Linux Streaming Telemetry Lab

SR Linux has first-class Streaming Telemetry support thanks to 100% YANG coverage of state and config data. The holistic coverage enables SR Linux users to stream **any** data off of the NOS with on-change, sample, or target-defined support. A discrepancy in visibility across APIs is not about SR Linux.

This lab represents a small Clos fabric with Nokia SR Linux switches running as containers. The lab topology consists of a Clos topology, plus a Streaming Telemetry stack comprised of gnmic, prometheus and grafana applications.

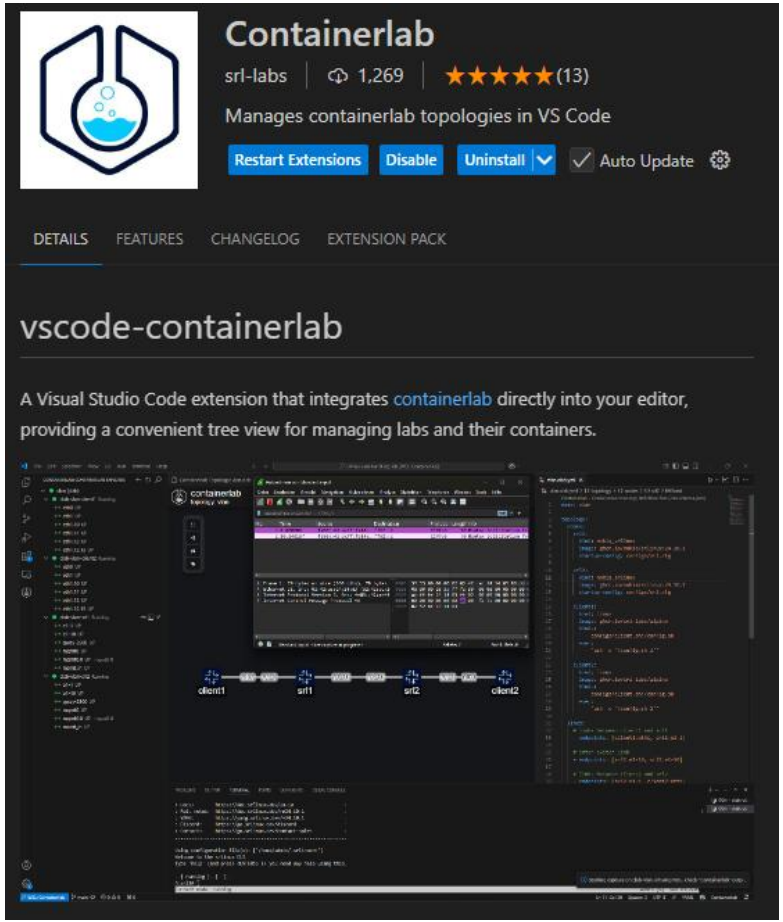
Run In Codespaces

Run this lab in GitHub Codespaces for free.
Learn more about Containerlab for Codespaces.

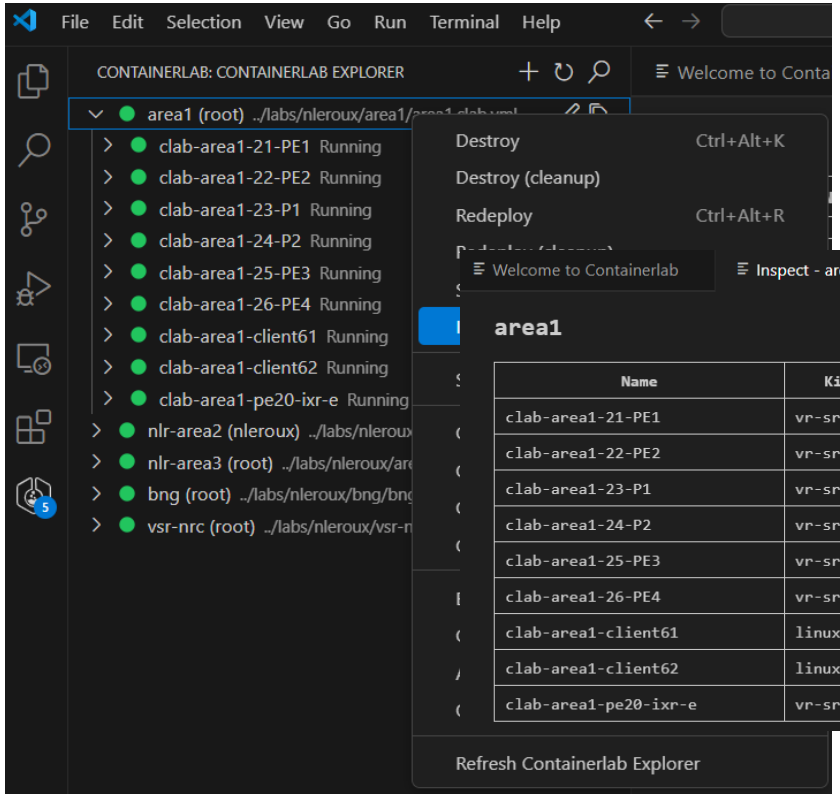
What's new?
VSCode-Containerlab plugin

- Discovers clab.yml files
- Allows to edit clab.yml files
- deploy/destroy/inspect labs
- Interact with labs
- Wireshark packet captures
- Visualize labs with topology views and draw.io

VS Code extension video



VSCode-Containerlab plugin
Lab discovery & actions

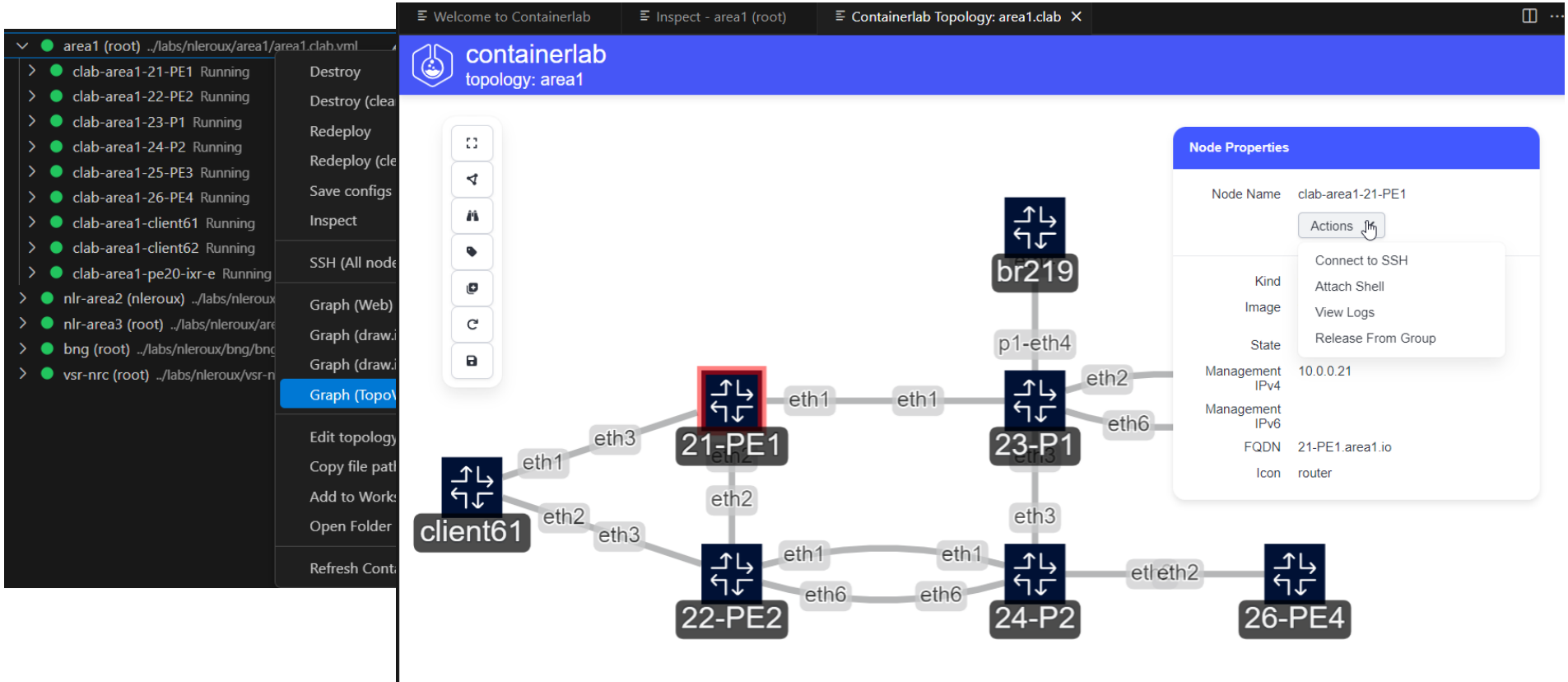


Welcome to Containerlab | Inspect - area1 (root)

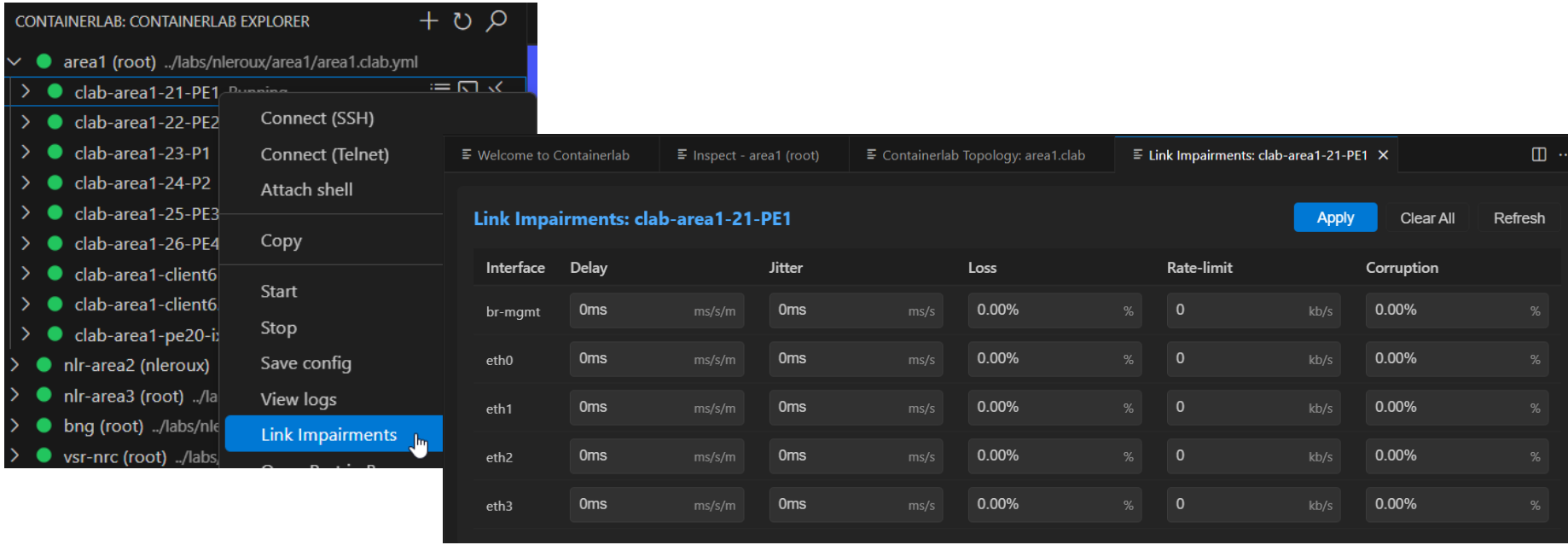
area1

Name	Kind	Image	State	IPv4	IPv6
clab-area1-21-PE1	vr-sros	vrnetlab/vr-sros:23.10.R5-bofpersist	running	10.0.0.21/24	N/A
clab-area1-22-PE2	vr-sros	vrnetlab/vr-sros:23.10.R5-bofpersist	running	10.0.0.22/24	N/A
clab-area1-23-P1	vr-sros	vrnetlab/vr-sros:23.10.R5-bofpersist	running	10.0.0.23/24	N/A
clab-area1-24-P2	vr-sros	vrnetlab/vr-sros:23.10.R5-bofpersist	running	10.0.0.24/24	N/A
clab-area1-25-PE3	vr-sros	vrnetlab/vr-sros:23.10.R5-bofpersist	running	10.0.0.25/24	N/A
clab-area1-26-PE4	vr-sros	vrnetlab/vr-sros:23.10.R5-bofpersist	running	10.0.0.26/24	N/A
clab-area1-client61	linux	ghcr.io/srl-labs/network-multitool:latest	running	10.0.0.61/24	N/A
clab-area1-client62	linux	ghcr.io/srl-labs/network-multitool:latest	running	10.0.0.62/24	N/A
clab-area1-pe20-ixr-e	vr-sros	vrnetlab/vr-sros:23.10.R5-bofpersist	running	10.0.0.20/24	N/A

VSCode-Containerlab plugin
Lab visualization and interaction



VSCode-Containerlab plugin
Link impairments



VSCode-Containerlab plugin
Capture interfaces with remote Wireshark output

CONTAINERlab: CONTAINERLAB EXPLORER

- area1 (root) ../labs/nleroux/area1/area1.clab.yml
  - clab-area1-21-PE1 Running
  - clab-area1-22-PE2 Running
    - br-mgmt UP
    - eth0 UP
    - eth1 UP
    - eth2 UP
    - eth3 UP
    - eth6 UP
  - clab-area1-23-P1 Running
  - clab-area1-24-P2 Running
  - clab-area1-25-PE3 Running
  - clab-area1-26-PE4 Running
  - clab-area1-client61 Running
  - clab-area1-client62 Running
  - clab-area1-pe20-ixr-e Running
- nlr-area2 (nleroux) ../labs/nleroux/area2/area2.clab.yml

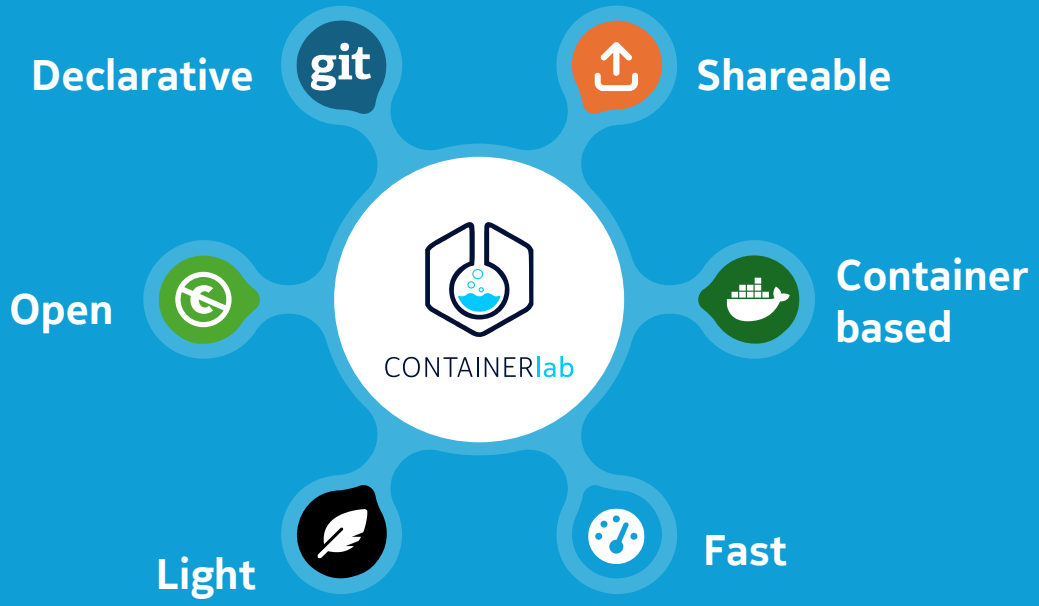
Capturing from packetflix:// remote cluster and container host capture (0.10.7)

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
15	6.026237936	192.168.0.206	224.0.0.2	LDP	121	Hello Message
16	6.969382508	192.168.0.207	224.0.0.5	OSPF	82	Hello Packet
17	6.999479101	192.0.2.24	192.168.0.212	RSVP	254	PATH Message. SESSION: IPv4-LSP, Destination 192.168.0.212, Short Call ID 0
18	7.706232609	192.168.0.206	192.168.0.207	RSVP	60	HELLO Message.
19	7.707163742	192.168.0.207	192.168.0.206	RSVP	60	HELLO Message.
20	7.926330646	192.168.0.206	224.0.0.5	OSPF	82	Hello Packet
21	9.469167481	192.168.0.207	224.0.0.2	LDP	121	Hello Message
22	10.026019642	192.168.0.206	224.0.0.2	LDP	121	Hello Message
23	10.706118306	192.168.0.206	192.168.0.207	RSVP	60	HELLO Message.
24	10.706124806	192.168.0.206	192.168.0.207	RSVP	178	RESV Message. SESSION: IPv4-LSP, Destination 192.168.0.212, Short Call ID 0
25	10.706959059	192.168.0.207	192.168.0.206	RSVP	60	HELLO Message.
26	11.706306832	192.0.2.21	192.168.0.211	RSVP	270	PATH Message. SESSION: IPv4-LSP, Destination 192.168.0.211, Short Call ID 0
27	11.999513680	192.168.0.207	192.168.0.206	RSVP	162	RESV Message. SESSION: IPv4-LSP, Destination 192.168.0.207, Short Call ID 0
28	12.209584970	192.0.2.24	192.0.2.22	LDP	84	Keep Alive Message
29	12.666093510	192.0.2.22	192.0.2.24	LDP	84	Keep Alive Message
30	12.709036110	192.0.2.24	192.0.2.22	TCP	66	51050 → 646 [ACK] Seq=37 Ack=37 Win=65535 Len=0 TSval=1209159395 TSecr=1040

```
> Frame 1: 194 bytes on wire (1552 bits), 194 bytes captured (1552 bits) on i
> Ethernet II, Src: 52:54:00:a9:28:41 (52:54:00:a9:28:41), Dst: 52:54:00:77:0
> Internet Protocol Version 4, Src: 192.168.0.207, Dst: 192.168.0.206
> Resource ReserVation Protocol (RSVP): RESV Message. SESSION: IPv4-LSP, Dest

0000  52 54 00 77 0a 41 52 54  00 a9 28 41 08 00 45 c0   RT·w·ART ··(A··E·
0010  00 b4 ce 13 00 00 ff 2e  69 5a c0 a8 00 cf c0 a8   ........ iZ····
0020  00 ce 10 02 a9 57 ff 00  00 a0 00 10 01 07 c0 00   ·····W·· ........
0030  02 19 00 00 00 03 c0 00  02 15 00 0c 03 01 c0 a8   ........ ........
0040  00 cf 00 00 00 02 00 08  05 01 00 00 75 30 00 08   ........ ····u0··
0050  08 01 00 00 00 12 00 24  09 02 00 00 00 07 05 00   ·......$ ........
0060  00 06 7f 00 00 05 00 00  00 00 00 00 00 00 7f 80   ........ ........
0070  00 00 00 00 00 14 00 00  1f 32 00 0c 0a 07 c0 00   ........ ·2......
0080  02 15 00 00 aa 02 00 08  10 01 00 07 ff f0 00 34   ........ .......4
0090  15 01 01 08 c0 a8 00 cf  20 00 03 08 01 01 00 07   ........ ........
00a0  ff f0 01 08 c0 a8 00 d1  20 00 03 08 01 01 00 07   ........ ........
00b0  ff f4 01 08 c0 a8 00 d3  20 00 03 08 01 01 00 07   ........ ........
00c0  ff e3                                               ··
```

Containerlab
Key take aways



576K
Downloads

170+
Contributors

50+
Supported NOSes

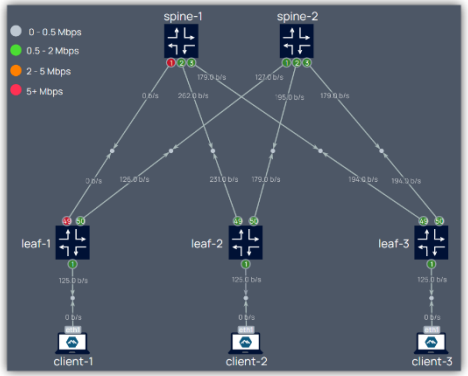
100+
Labs

They use Containerlab

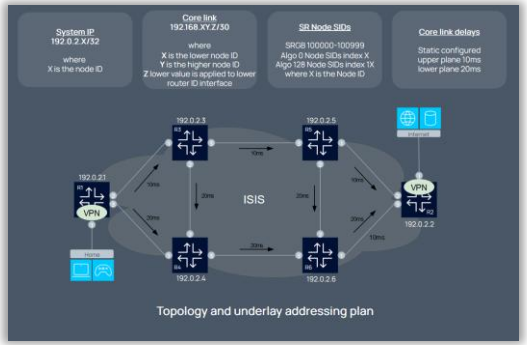
NOKIA
cisco
ARISTA
Juniper NETWORKS
Google
EQUINIX
redhat
Apple
The New York Times
cilium
netbox
NANOG
PACKETFABRIC
Proton
>>> network .toCode()
KEYSIGHT
GAMMA
OVHcloud
linx
telenet
DE-CIX
orange

Containerlab available labs

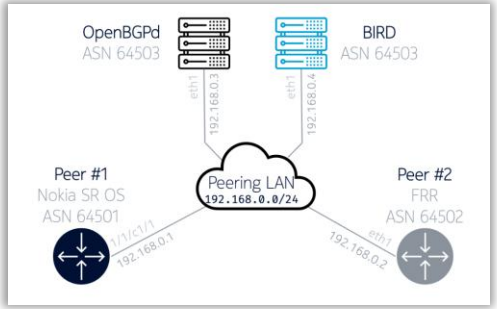
SR Linux Telemetry



Segment Routing



Peering Lab



Containerlab
Get in touch

CONTAINERlab

https://containerlab.dev

Discord server

srl-labs/containerlab

CONTAINERlab

Thank you